

Python Dictionaries

Bonus — 10 Uncommon Methods Worth Knowing

Focus	Uncommon but powerful dictionary methods and extensions
Includes	operator · = · Counter · defaultdict · OrderedDict · ChainMap · zip() · **unpacking · reversed()
Constraint	No loops or conditionals — pure dictionary and built-in method usage
Bonus	Each question includes a Why This Matters note for interview context

B1 Merge two dictionaries using the | operator

BONUS Method: | merge operator (Python 3.9+)

Two configuration dictionaries need to be merged into one. Use the | operator to combine them in a single expression without modifying either original. If a key exists in both, the right-hand dictionary wins.

WHY THIS MATTERS

Before Python 3.9, merging required `.update()` or unpacking (`**`). The | operator is cleaner, more readable, and works like addition — it is now the preferred way in modern Python.

INPUT / SETUP

```
defaults = {"theme": "light", "lang": "en",  
            "timeout": 30}  
user_cfg = {"theme": "dark", "font": "Arial"}
```

EXPECTED OUTPUT

```
{"theme": "dark", "lang": "en", "timeout": 30,  
 "font": "Arial"}
```

B2 Update a dictionary in-place using |=

BONUS

Method: |= update operator (Python 3.9+)

A settings dictionary needs to be updated with new values from a patch dictionary. Use the |= operator to apply the patch directly onto the original dictionary in one line.

WHY THIS MATTERS

The |= operator is the in-place version of |. It is the modern replacement for .update() — shorter and more expressive. Interviewers may ask the difference between | and |=.

INPUT / SETUP

```
settings = {"volume": 70, "brightness": 80,
            "wifi": True}
patch = {"brightness": 50, "bluetooth": True}
```

EXPECTED OUTPUT

```
{"volume": 70, "brightness": 50, "wifi": True,
 "bluetooth": True}
```

B3 Count character frequency using Counter

BONUS

Method: collections.Counter

Given a word, count how many times each character appears. Use Counter from the collections module — it returns a dictionary-like object where keys are characters and values are their counts.

WHY THIS MATTERS

Counter is a subclass of dict built exactly for counting. It eliminates the need for a loop + manual counting. It is extremely common in interview problems and data processing tasks.

INPUT / SETUP

```
from collections import Counter
word = "mississippi"
```

EXPECTED OUTPUT

```
Counter({'s': 4, 'i': 4, 'p': 2, 'm': 1})
```

B4 Find the two most common words

BONUS

Method: `collections.Counter + .most_common()`

Given a list of words, find the 2 most frequently occurring ones along with their counts. Use `Counter` with `.most_common(n)` to retrieve the top `n` items.

WHY THIS MATTERS

`.most_common()` returns items sorted by frequency — highest first. It avoids sorting manually and is very efficient. Widely used in text analysis and NLP preprocessing.

INPUT / SETUP

```
from collections import Counter
words = ["apple", "banana", "apple", "cherry",
         "banana", "apple", "cherry", "banana"]
```

EXPECTED OUTPUT

```
[("apple", 3), ("banana", 3)]
```

B5 Group items without `KeyError` using `defaultdict`

BONUS

Method: `collections.defaultdict`

Given a list of (student, subject) tuples, group all subjects per student into a dictionary of lists. Use `defaultdict(list)` so you never get a `KeyError` when a student appears for the first time.

WHY THIS MATTERS

A regular dict raises `KeyError` if you try to append to a key that does not exist yet. `defaultdict` automatically creates the default value so grouping becomes trivial — no `.get()` workarounds needed.

INPUT / SETUP

```
from collections import defaultdict
data = [("Ali", "Math"), ("Mia", "Science"),
        ("Ali", "English"), ("Mia", "Math")]
```

EXPECTED OUTPUT

```
{"Ali": ["Math", "English"], "Mia": ["Science",
                                     "Math"]}
```

B6 Maintain insertion order with OrderedDict

BONUS

Method: `collections.OrderedDict + .move_to_end()`

Create an `OrderedDict` of tasks. Then use `.move_to_end()` to move the task 'email' to the front of the order (`last=False` means front). Print the final order of keys.

WHY THIS MATTERS

Regular dicts maintain insertion order since Python 3.7, but `OrderedDict` has extra tools like `.move_to_end()` and `.popitem(last=False)` that regular dicts do not have. Useful for LRU caches and priority queues.

INPUT / SETUP

```
from collections import OrderedDict
tasks = OrderedDict([("standup", "done"),
("email", "pending"),
("review", "done")])
tasks.move_to_end("email", last=False)
```

EXPECTED OUTPUT

```
OrderedDict([("email", "pending"),
("standup", "done"),
("review", "done")])
```

B7 Search multiple dictionaries as one using ChainMap

BONUS

Method: `collections.ChainMap`

You have a local config and a global config. Use `ChainMap` to create a combined view where local values take priority over global ones. Look up 'timeout' and 'retries' from the combined map.

WHY THIS MATTERS

`ChainMap` lets you search multiple dicts as a single unit without merging them. Changes to the `ChainMap` affect only the first dict. Used in Python's own variable scope resolution (locals → globals → builtins).

INPUT / SETUP

```
from collections import ChainMap
local_cfg = {"timeout": 10, "debug": True}
global_cfg = {"timeout": 30, "retries": 5,
"debug": False}
combined = ChainMap(local_cfg, global_cfg)
```

EXPECTED OUTPUT

```
combined["timeout"] -> 10 (local wins)
combined["retries"] -> 5 (from global)
```

B8 Build a dictionary from two lists using zip()

BONUS

Method: zip() + dict()

Given two separate lists — one of keys and one of values — combine them into a single dictionary using zip() and dict(). This is a common pattern when reading data from CSV columns or API responses.

WHY THIS MATTERS

zip() pairs up items from two iterables positionally. Wrapping it in dict() instantly turns paired tuples into a dictionary. Far cleaner than manually indexing two lists.

INPUT / SETUP

```
keys = ["name", "age", "city", "score"]
values = ["Ali", 22, "Toronto", 91]
```

EXPECTED OUTPUT

```
{"name": "Ali", "age": 22, "city": "Toronto",
"score": 91}
```

B9 Unpack a dictionary into a function call using **

BONUS

Method: ** dictionary unpacking

A function greet() accepts name, age, and city as keyword arguments. You have these values stored in a dictionary. Use ** unpacking to pass the dictionary directly into the function without listing each argument.

WHY THIS MATTERS

** unpacking turns a dictionary into keyword arguments in one go. It is used extensively in real code — passing config dicts to functions, constructing objects, and merging dicts (the pre-3.9 way).

INPUT / SETUP

```
def greet(name, age, city):
    print(f"Hi {name}, age {age}, from {city}!")
info = {"name": "Mia", "age": 23, "city":
"Vancouver"}
greet(**info)
```

EXPECTED OUTPUT

```
Hi Mia, age 23, from Vancouver!
```

B10 Iterate a dictionary in reverse order

BONUS

Method: `reversed()` on dict (Python 3.8+)

Given a dictionary of student ranks, iterate through it in reverse insertion order using `reversed()`. Print each key from last inserted to first.

WHY THIS MATTERS

Before Python 3.8, dicts could not be reversed directly. Now `reversed()` works on dicts just like on lists. Useful when you need LIFO (last-in, first-out) access without converting to a list first.

INPUT / SETUP

```
ranks = {"Ali": 1, "Mia": 2, "Joe": 3, "Eva": 4}
list(reversed(ranks))
```

EXPECTED OUTPUT

```
["Eva", "Joe", "Mia", "Ali"]
```

Python Dictionary Bonus Questions · These methods will set your students apart.